

Gold artifact – Architecture

Case study: Appointment Booking System

1. Cel architektury

Zaprojektowanie spójnego systemu do:

- przeglądania terminów,
- rezerwacji wizyt,
- anulowania wizyt,
- zarządzania grafiką specjalisty,
- obsługi konfliktów czasowych,
- rejestrowania historii operacji.

Architektura ma wspierać:

- brak double booking,
- limit 3 aktywnych rezerwacji na użytkownika,
- wykrywanie nakładających się wizyt,
- kontrolę dostępu do danych,
- obsługę dodatkowego stanu slotu BLOCKED,
- audyt operacji,
- prostą implementację i testowanie.

2. Styl architektury

- Przyjęty styl:
- modular monolith
- REST API
- relacyjna baza danych
- Powód:
- wystarczający dla skali case study,
- prosty do implementacji,
- łatwy do testowania i wdrożenia,
- pozwala kontrolować spójność danych,

- umożliwia wydzielenie logiki biznesowej do osobnych usług.

3. Główne komponenty

C1. API Layer

- Odpowiada za:
- obsługę żądań HTTP,
- walidację danych wejściowych,
- przekazanie żądań do warstwy logiki.

C2. Booking Service

- Odpowiada za:
- rezerwację slotu,
- anulowanie rezerwacji,
- sprawdzanie limitu 3 aktywnych rezerwacji,
- kontrolę właściciela rezerwacji,
- koordynację logiki biznesowej między innymi usługami.

C3. Schedule Service

- Odpowiada za:
- dodawanie slotów przez specjalistę,
- usuwanie wolnych slotów,
- pobieranie dostępnych terminów,
- sprawdzanie dostępności slotu,
- wykrywanie konfliktów czasowych między wizytami użytkownika,
- zmianę statusów slotów.

C4. Audit Service

- Odpowiada za:
- zapis zdarzeń biznesowych,
- rejestrowanie rezerwacji, anulowań i odrzuconych operacji,
- udostępnianie historii operacji.

C5. User/Access Service

- Odpowiada za:
- identyfikację roli użytkownika,
- kontrolę dostępu do rezerwacji i slotów,
- obsługę uprawnień użytkownika, specjalisty i admina.

C6. Persistence Layer

- Odpowiada za:
- zapis i odczyt danych z bazy,
- utrzymanie encji domenowych,
- obsługę trwałości rezerwacji, slotów i logów audytowych,
- wsparcie transakcyjnej obsługi rezerwacji.

4. Model danych

- User
- id
- name
- role (USER, SPECIALIST, ADMIN)
- Specialist
- id
- user_id
- specialization
- Slot
- id
- specialist_id
- start_time
- end_time
- status (AVAILABLE, BOOKED, BLOCKED, COMPLETED)
- Booking
- id
- user_id

- slot_id
- status (BOOKED, CANCELLED)
- created_at
- AuditLog
- id
- event_type
- user_id
- slot_id
- timestamp
- details

5. Relacje między komponentami

API Layer wywołuje:

- Booking Service
- Schedule Service
- User/Access Service
- Audit Service (pośrednio przez warstwę logiki lub endpoint administracyjny)

Booking Service korzysta z:

- Schedule Service
- Audit Service
- Persistence Layer
- User/Access Service

Schedule Service korzysta z:

- Persistence Layer

Audit Service korzysta z:

- Persistence Layer

User/Access Service korzysta z:

- Persistence Layer

6. Główne endpointy API

User

- GET /slots?specialist_id=&date=
- POST /bookings
- DELETE /bookings/{id}
- GET /bookings/my

Specialist

- POST /slots
- DELETE /slots/{id}
- PATCH /slots/{id}/block
- GET /slots/my

Admin

- GET /users
- GET /bookings
- GET /audit-log

7. Kluczowe decyzje architektoniczne

D1. Modular monolith zamiast microservices

Przyjęto jeden system z wyraźnym podziałem logicznym na moduły.

Powód: mniejsza złożoność, łatwiejsza implementacja i ocena.

D2. Relacyjna baza danych

Przyjęto bazę SQL.

Powód: łatwiejsza kontrola spójności i transakcji przy rezerwacjach, konfliktach czasowych i historii operacji.

D3. Transakcyjna rezerwacja slotu

Operacja rezerwacji musi być atomowa.

Powód: zapobieganie double booking przy równoczesnych żądaniach.

D4. Wydzielenie Schedule Service

Sprawdzanie dostępności slotu i konfliktów czasowych znajduje się poza Booking Service.

Powód: lepszy podział odpowiedzialności i prostsze testowanie.

D5. Wydzielenie Audit Service

Rejestrowanie operacji jest osobnym komponentem.

Powód: oddzielenie logiki biznesowej od logiki audytu.

D6. Jawna kontrola ról

Dostęp do operacji zależy od roli użytkownika.

Powód: bezpieczeństwo i zgodność z wymaganiami.

D7. Dodatkowy stan slotu BLOCKED

Slot może być zablokowany niezależnie od zwykłego cyklu rezerwacji.

Powód: odzwierciedlenie bardziej złożonych reguł biznesowych.

8. Przeptyw rezerwacji

Użytkownik wysyła żądanie rezerwacji slotu.

API Layer waliduje format danych.

Booking Service:

1. sprawdza uprawnienia użytkownika,
2. pobiera slot przez Schedule Service,
3. sprawdza, czy slot jest dostępny,
4. sprawdza limit aktywnych rezerwacji,
5. sprawdza konflikt czasowy z innymi wizytami użytkownika,
6. uruchamia transakcję,
7. zapisuje rezerwację,
8. zmienia status slotu na BOOKED,
9. zapisuje zdarzenie w Audit Service.
10. System zwraca wynik operacji.

9. Przeptyw anulowania

Użytkownik wysyła żądanie anulowania.

Booking Service:

1. sprawdza właściciela rezerwacji,
2. sprawdza warunek 24h,
3. zmienia status booking na CANCELLED,
4. pobiera powiązany slot,
5. przywraca slot do AVAILABLE lub pozostawia go w BLOCKED, zależnie od aktualnego stanu,
6. zapisuje zdarzenie w Audit Service.
 - o System zwraca wynik operacji.

10. Przeptyw wykrywania konfliktu czasowego

1. Użytkownik wybiera slot do rezerwacji.

2. Booking Service przekazuje żądanie do Schedule Service.
3. Schedule Service pobiera aktywne rezerwacje użytkownika.
4. Dla każdej aktywnej rezerwacji porównywane są przedziały czasowe.
5. Jeśli wykryto nakładanie się przedziałów, operacja rezerwacji zostaje odrzucona.
6. Zdarzenie odrzucenia zostaje zapisane w Audit Service.

11. Ograniczenia przyjęte w gold

W tej architekturze przyjęto, że:

- brak integracji z płatnościami,
- brak powiadomień e-mail/SMS,
- brak obsługi stref czasowych,
- brak rozproszonej architektury,
- jeden backend i jedna baza danych,
- brak pełnego mechanizmu kolejkowania lub asynchronicznego przetwarzania zdarzeń.

11. Kryteria jakości tej architektury

Ta wersja referencyjna powinna spełniać:

- zgodność z bardziej złożonymi requirements,
- modularność opartą na osobnych odpowiedzialnościach,
- możliwość implementacji bez nadmiarowej infrastruktury,
- łatwość testowania logiki rezerwacji, konfliktów i audytu,
- odporność na podstawowe konflikty rezerwacji,
- spójność między stanami bookingów, slotów i logów.

12. Do czego używać tego gold

Ten artifact może być użyty jako:

- gold architecture dla Architecture Team,
- wejście lokalne dla Coding Team,
- punkt odniesienia do oceny architektur generowanych przez studentów i LLM.